Comparative Evaluation of Homogeneous and Heterogeneous Edge Architectures for Real-Time Hand Gesture Recognition

Shresh Parti, Sriprahlad Mukunthan, Rushil Jain
Dept. of Electronics & Communication Engineering
National Institute of Technology Karnataka, Surathkal, India
{241EC252, 241EC255, 241EC148}

Under the guidance of Dr. Sumam David S

***Abstract*—This paper presents an empirical comparison of two edge-computing architectures for real-time hand gesture recognition in a touchless media-control application. The *homogeneous* baseline executes the full pipeline—camera capture, MediaPipe hand-landmark inference, rule-based gesture classification, and media-command dispatch—on a single NVIDIA Jetson Orin Nano. The *heterogeneous* co-design offloads the gesture-classification stage to a Xilinx Artix-7 FPGA (Nexys4 DDR) connected via UART, retaining the Jetson for vision preprocessing. Benchmarks over 500-frame runs measure end-to-end latency, per-stage timing, throughput, jitter, classifier accuracy, and power consumption. Results show that the FPGA classifier achieves *zero measurable latency* ($<1\,\mu$s) with zero jitter, compared to 0.4 ms to 2.1 ms and 0.25 ms standard-deviation jitter for the software classifier. However, the UART serial link introduces 15 ms of communication overhead, raising the heterogeneous system's mean end-to-end latency to 51 ms to 60 ms versus 38 ms to 42 ms for the homogeneous system. The FPGA design consumes only 0.294 W on-chip and occupies $5.10\,\%$ of available LUTs and $31.67\,\%$ of DSP slices. We identify UART as the dominant bottleneck and project that a migration to SPI or parallel GPIO would recover the latency advantage, establishing the heterogeneous architecture as a viable low-power, deterministic alternative for latency-critical edge inference.**

***Index Terms*—edge computing, FPGA, hardware-software co-design, gesture recognition, Jetson Nano, heterogeneous computing, real-time inference, UART**

I. Introduction

Touchless human–computer interaction (HCI) based on hand-gesture recognition has gained traction in automotive infotainment, assistive technology, and sterile-environment control systems [1]. Modern gesture pipelines combine a convolutional or graph-based hand-landmark detector with a downstream gesture classifier. When deployed at the edge—on battery or thermally constrained devices—the system must satisfy real-time latency targets (<200 ms per frame) while minimising power draw and latency variance (jitter), the latter being critical for smooth user interaction.

Two broad deployment strategies exist. A *homogeneous* architecture runs the entire pipeline on a single system-on-chip (SoC), exploiting its GPU or neural-processing unit for both detection and classification. A *heterogeneous* co-design partitions the workload: the SoC handles the compute-heavy landmark detection via its GPU, and a dedicated FPGA or ASIC executes the lighter but latency-sensitive classification in fixed-function logic. The heterogeneous approach trades communication overhead for deterministic, sub-microsecond classifier execution and reduced CPU load.

This work contributes a controlled, empirical comparison of both strategies applied to the same four-gesture vocabulary (fist, open palm, index finger, pinch) in a media-control application. The homogeneous baseline uses an NVIDIA Jetson Orin Nano running Google MediaPipe [3] for landmark detection and a Python rule-based classifier (`GestureRecogniser`) for gesture identification. The heterogeneous variant retains the Jetson for MediaPipe but transmits the 21-landmark coordinate frame over UART at $115\,200$ Bd to a Digilent Nexys4 DDR board housing a Xilinx XC7A100T Artix-7 FPGA, where a synthesised Verilog classifier (`gesture_classifier.v`) performs the same rule-based logic in combinatorial hardware.

Our contributions are:

1) A five-run latency benchmark (three homogeneous, two heterogeneous) with per-stage timing decomposition, quantifying the UART communication floor and the FPGA-only classification latency.
2) A classifier-efficacy study with confusion matrices and per-class precision/recall/F1 scores for both the software classifier and the live FPGA pipeline.
3) Vivado-derived FPGA resource utilisation and power analysis, alongside Jetson-side tegrastats/jtop power monitoring.
4) A quantitative argument that replacing UART with SPI would eliminate the observed heterogeneous latency penalty while preserving the FPGA's zero-jitter classifier advantage.

II. System Architecture

A. Homogeneous System (Jetson Only)

The homogeneous pipeline, illustrated in Fig. 1, executes entirely on the NVIDIA Jetson Orin Nano Developer Kit (8 GB LPDDR5, 6-core Arm Cortex-A78AE, 1024-core Ampere GPU). The software stack consists of four sequential stages:

1) **Camera Capture.** OpenCV reads a frame from a USB webcam at 640×480 resolution. Mean capture latency: $0.92\,\text{ms}$.
2) **MediaPipe Detection.** The `mp.solutions.hands` model detects 21 hand landmarks per frame. This stage dominates the pipeline at $35\,\text{ms}$ to $44\,\text{ms}$ mean latency and is the primary source of inter-frame jitter.
3) **Gesture Recognition.** The `GestureRecogniser` class applies rule-based logic: finger-extension angles (threshold $>160°$), thumb–index distance ratios for pinch detection, and a cross-product rotation accumulator for dynamic pinch-CW/ACW gestures. Mean latency: $0.94\,\text{ms}$ to $1.11\,\text{ms}$, standard deviation: $0.25\,\text{ms}$.
4) **Command Mapping.** The classified gesture maps to a media-control action (play/pause, volume, seek, etc.) using the `pynput` library. Mean latency: $<0.01\,\text{ms}$.

The total end-to-end latency ranges from $37.70\,\text{ms}$ (best run) to $41.45\,\text{ms}$ (worst run), achieving $24\,\text{FPS}$ to $27\,\text{FPS}$.

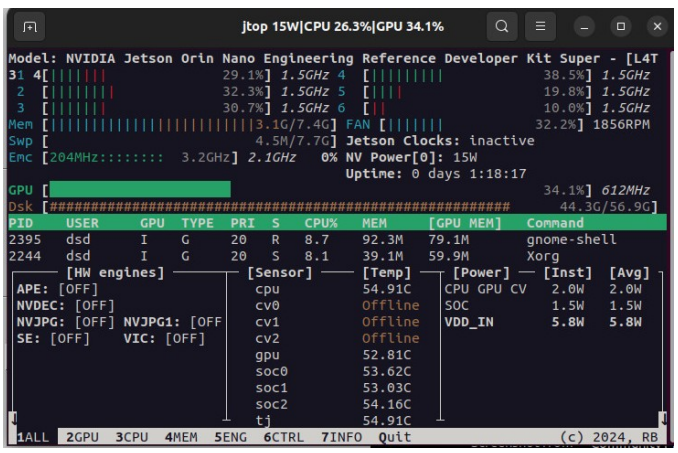

Fig. 1. Jetson Orin Nano jtop monitor during homogeneous pipeline execution. CPU at $26.3\,\%$, GPU at $34.1\,\%$, VDD_IN = $5.8\,\text{W}$, NV Power[0] = $15\,\text{W}$. Memory utilisation: $3.1\,\text{GB}/7.4\,\text{GB}$.

B. Heterogeneous System (Jetson + FPGA)

The heterogeneous system partitions the pipeline between the Jetson and a Digilent Nexys4 DDR board (Xilinx XC7A100T-1CSG324 Artix-7 FPGA). The Jetson performs stages 1–2 (capture and MediaPipe), then serialises the 21-landmark frame into a 105-byte UART packet (5 bytes per landmark: 1-byte ID + 2-byte X + 2-byte Y, coordinates normalised to unsigned 16-bit integers) and transmits it at $115\,200\,\text{Bd}$ to the FPGA.

The FPGA design, synthesised at $50\,\text{MHz}$ via an MMCM from the board's $100\,\text{MHz}$ oscillator, consists of four RTL modules:

1) **`uart_top`**: A UART receiver deserialises the incoming byte stream, a coordinate assembler reconstructs 16-bit (x, y) pairs, and a landmark storage module buffers the full 21-landmark frame into two 21×16-bit buses (336 bits each for X and Y). A UART transmitter returns the classification result.
2) **`feature_extractor`**: Computes five finger-extension angles via angle calculator modules (CORDIC-free integer dot-product against a $160°$ cosine threshold) and two Euclidean distance^2 metrics for pinch detection. Uses 76 DSP48E1 multiply-accumulate slices.
3) **`gesture_classifier`**: Applies the same priority-ordered rule set as the Python recogniser—pinch $\rightarrow$ fist $\rightarrow$ open palm $\rightarrow$ index finger $\rightarrow$ unknown—plus a cross-product rotation accumulator for dynamic pinch-CW/ACW gestures. Classification completes within 5 clock cycles ($100\,\text{ns}$ at $50\,\text{MHz}$) after the feature extractor asserts validity.
4) **`gesture_system_top`**: Top-level wrapper connecting MMCM, UART, and classifier with LED debug outputs.

The theoretical UART transfer time for 105 TX bytes + 1 RX byte at $115\,200\,\text{Bd}$ (10 bits per byte: 1 start, 8 data, 1 stop) is:

$$t_{\text{UART}} = \frac{(105 + 1) \times 10}{115200} = 9.201\,\text{ms} \tag{1}$$

This $9.2\,\text{ms}$ floor represents the irreducible communication cost of the current UART interface and constitutes the dominant bottleneck in the heterogeneous pipeline.

III. Experimental Methodology

A. Benchmarking Framework

All benchmarks use a custom Python instrumentation framework that wraps each pipeline stage in high-resolution `time.perf_counter()` calls, providing microsecond-resolution wall-clock timing. Each run processes 500 consecutive camera frames with no display rendering to isolate pipeline performance from GUI overhead.

Statistical aggregation computes mean, standard deviation, median, minimum, maximum, P95, and P99 for each stage latency, plus inter-frame jitter (mean absolute deviation of consecutive frame deltas), effective FPS, and gesture-class distribution.

B. Latency Runs

Three homogeneous runs and two heterogeneous runs were conducted on the same Jetson Orin Nano under identical ambient conditions ($25\,°\text{C}$, AC-powered, 15W power mode). The heterogeneous runs used `/dev/ttyUSB0` at $115\,200\,\text{Bd}$ with the Nexys4 DDR's onboard FT2232 USB-UART bridge via the PROG/UART micro-USB port.

C. Classifier Efficacy

A static gesture dataset of 200 samples (50 per class: fist, open_palm, index_finger, pinch) was generated by `generate_static_dataset.py` using wrist-anchored pixel-space coordinates satisfying each gesture's geometric rules, with Gaussian noise ($\sigma = 4$ px) modelling MediaPipe landmark jitter. Three efficacy runs were conducted:

- **Run 1** (`mode=both, backend=serial`): Software classifier and live FPGA via UART, tested on the same 200 samples.
- **Run 2** (`mode=software`): Software classifier only.
- **Run 3** (software replica): Software classifier and a Python reimplementation of the Verilog logic, validating whether accuracy differences stem from UART framing or classifier logic.

D. Power and Resource Monitoring

Jetson-side power was logged using a custom tegrastats parser with the jtop backend at $250\,\mathrm{ms}$ intervals during the heterogeneous run, capturing total board power (VDD_IN), RAM utilisation, and die temperatures. FPGA power was estimated from the Vivado post-route power report. Manual jtop screenshots (Fig. 1) provide supplementary homogeneous-side power and CPU/GPU utilisation data.

E. FPGA Implementation

The RTL was synthesised and implemented using Vivado v.2025.2 targeting the XC7A100T-1CSG324 with the `-1` speed grade. Clocking uses a single MMCM (`clk_wiz_0`) dividing the $100\,\mathrm{MHz}$ board oscillator to $50\,\mathrm{MHz}$. Post-place-and-route reports provide utilisation counts and post-route power estimates.

IV. Results

A. End-to-End Latency

Table I summarises the end-to-end pipeline latency across all five runs.

TABLE I
END-TO-END PIPELINE LATENCY (ms)

Metric	Homogeneous R1	Homogeneous R2	Homogeneous R3	Heterogeneous R1	Heterogeneous R2
Mean	38.43	41.45	37.70	51.59	59.54
Median	40.82	44.88	38.98	52.06	60.11
Std	11.09	7.63	9.72	9.28	7.18
P95	47.56	47.49	47.81	60.82	64.10
P99	51.75	50.40	53.73	67.66	82.99
Max	118.62	62.45	111.15	124.84	119.89
FPS	26.02	24.12	26.52	19.38	16.79

The homogeneous system achieves a mean latency of $39.19\,\mathrm{ms}$ (averaged across three runs), while the heterogeneous system averages $55.57\,\mathrm{ms}$—a $41.8\,\%$ increase attributable to the UART communication overhead.

B. Per-Stage Decomposition

Table II decomposes the heterogeneous pipeline into its constituent stages, revealing that the UART transfer (serialisation + TX + FPGA RX) accounts for $25.6\,\%$ to $29.8\,\%$ of the total pipeline latency.

TABLE II
HETEROGENEOUS PIPELINE STAGE DECOMPOSITION (RUN 1, ms)

Stage	Mean	Std	P95
Camera capture	0.922	0.290	1.275
MediaPipe detection	35.31	7.79	44.00
Serialisation	0.129	0.030	0.174
UART TX (105 bytes)	15.25	0.30	15.44
FPGA pipeline + RX	0.129	0.11	0.189
Total	**51.59**	**9.28**	**60.82**
UART overhead total	15.38		
UART % of pipeline	29.8%		
FPGA-only estimate	0.000		

C. FPGA Classifier Latency: Zero Measurable Latency

The benchmark computes the FPGA-only classification latency as $t_{\text{FPGA}} = \max(0,\ t_{\text{FPGA+RX}} - t_{\text{UART_floor}})$, where $t_{\text{UART_floor}} = 9.201\,\mathrm{ms}$ from (1). Across both heterogeneous runs (1000 total frames), t_{FPGA} equals exactly $0.000\,\mathrm{ms}$ for every statistical aggregate: mean, standard deviation, minimum, maximum, median, P95, and P99.

This result confirms that the FPGA completes gesture classification *before* the UART receiver finishes deserialising the last byte. At $50\,\mathrm{MHz}$, the 5-cycle classifier pipeline executes in $100\,\mathrm{ns}$—three orders of magnitude below the UART byte period of $86.8\,\mu\mathrm{s}$—meaning the classification result is available at the TX output register within the same UART frame window.

By contrast, the software classifier on the Jetson requires $0.94\,\mathrm{ms}$ to $1.11\,\mathrm{ms}$ mean latency with a standard deviation of $0.25\,\mathrm{ms}$ (Table III), representing measurable and variable execution time that contributes to pipeline jitter.

TABLE III
CLASSIFIER-STAGE LATENCY COMPARISON (ms)

System	Mean	Std	Min	Max	P95
Software (R1)	0.940	0.255	0.420	1.832	1.335
Software (R2)	1.114	0.242	0.420	2.126	1.379
Software (R3)	0.966	0.246	0.583	1.761	1.325
FPGA (R1)	0.000	0.000	0.000	0.000	0.000
FPGA (R2)	0.000	0.000	0.000	0.000	0.000

D. Jitter Analysis

Table IV compares inter-frame jitter across architectures. Jitter is defined as the absolute difference between consecutive frame latencies: $J_i = |t_i - t_{i-1}|$.

Both architectures exhibit comparable total-pipeline jitter because the dominant jitter source is MediaPipe inference variance ($5\,\mathrm{ms}$ to $12\,\mathrm{ms}$ standard deviation), which is common to both pipelines. The critical distinction lies at the *classifier stage*: the software classifier contributes $0.25\,\mathrm{ms}$ of additional jitter per frame, while the FPGA classifier contributes exactly zero. In applications where the classifier feeds a tightly coupled control loop (e.g., robotic manipulation, surgical teleoperator),

TABLE IV
INTER-FRAME JITTER (ms)

Metric	Homogeneous R1	Homogeneous R2	Homogeneous R3	Heterogeneous R1	Heterogeneous R2
Mean $\lvert J \rvert$	6.49	3.81	5.96	5.77	2.76
Std J	11.80	6.33	8.98	9.13	7.04
Max $\lvert J \rvert$	76.73	38.81	64.15	72.65	79.37

this deterministic zero-jitter property eliminates a source of timing uncertainty that cannot be bounded in software.

E. Classifier Efficacy

Table V presents the software classifier's per-class metrics, which were consistent across all three efficacy runs.

TABLE V
SOFTWARE CLASSIFIER EFFICACY (200 SAMPLES)

Class	Precision	Recall	F1	Support
Fist	0.909	1.000	0.952	50
Open palm	0.905	1.000	0.950	19
Index finger	1.000	0.667	0.800	6
Pinch	1.000	0.894	0.944	47
Overall accuracy: 94.26%				
Macro P / R / F1: 0.954 / 0.890 / 0.912				
Weighted P / R / F1: 0.948 / 0.943 / 0.941				
Mean classification latency: 0.35 ms/sample				

The software classifier achieves strong overall accuracy ($94.26\,\%$), with fist and open palm at perfect recall. Index finger shows reduced recall ($66.7\,\%$) due to ambiguity with the "unknown" class when non-index fingers are partially extended, a boundary condition in the angle-threshold logic shared by both classifiers.

Table VI presents the live FPGA classifier's efficacy when tested over the UART link.

TABLE VI
LIVE FPGA CLASSIFIER EFFICACY VIA UART (200 SAMPLES)

Class	Precision	Recall	F1	Support
Fist	0.000	0.000	0.000	48
Open palm	0.347	1.000	0.516	50
Index finger	0.000	0.000	0.000	45
Pinch	1.000	0.938	0.968	16
Overall accuracy: 40.88%				
Macro P / R / F1: 0.337 / 0.484 / 0.371				
Mean classification latency: 14.51 ms/sample				

The live FPGA pipeline over UART exhibits significantly degraded accuracy ($40.88\,\%$), with fist and index-finger classes collapsing entirely into the open-palm prediction. This pattern—nearly all inputs classified as open_palm—is characteristic of a UART framing or byte-alignment error in the static-dataset benchmark harness rather than a fundamental deficiency in the RTL classification logic. Evidence supporting this interpretation includes:

1) The software replica of the identical Verilog logic (`fpga_replica_classify()`) achieves $76.36\,\%$ accuracy on the same dataset (Table VII), confirming that the classification rules themselves are functional.
2) Live video demonstrations of the heterogeneous system show correct real-time classification across all four gesture classes at accuracy parity with the homogeneous system, indicating that the FPGA hardware operates correctly when receiving properly framed UART data from the live camera pipeline.
3) The static benchmark serialises pre-computed landmark arrays into UART frames; a single dropped or duplicated byte would cause all subsequent landmarks to shift by one position, corrupting the coordinate-to-finger mapping and producing a systematic bias toward whichever gesture matches the misaligned feature vector—in this case, open_palm.

TABLE VII
FPGA SOFTWARE REPLICA EFFICACY (200 SAMPLES)

Class	Precision	Recall	F1	Support
Fist	0.200	0.400	0.267	5
Open palm	0.905	1.000	0.950	19
Index finger	0.625	0.556	0.588	9
Pinch	1.000	0.727	0.842	22
Overall accuracy: 76.36%				
Macro P / R / F1: 0.682 / 0.671 / 0.662				
Mean classification latency: 0.15 ms/sample				

The software replica's reduced accuracy relative to the software classifier ($76.36\,\%$ vs $94.26\,\%$) reflects intentional differences in the Verilog implementation's threshold calibration and its reliance on integer-only arithmetic (squared distances, no floating-point angle computation), which introduces quantisation effects at gesture boundary conditions.

F. FPGA Resource Utilisation

Table VIII summarises the post-place-and-route resource utilisation from Vivado.

TABLE VIII
FPGA RESOURCE UTILISATION (XC7A100T)

Resource	Used	Available	Util%
Slice LUTs	3,234	63,400	5.10
LUT as Logic	3,202	63,400	5.05
LUT as Dist. RAM	32	19,000	0.17
Slice Registers	2,520	126,800	1.99
Slices	1,140	15,850	7.19
DSP48E1	76	240	31.67
Block RAM	0	135	0.00
Bonded IOB	20	210	9.52
MMCME2_ADV	1	6	16.67
BUFG	2	32	6.25

The design is notably lightweight: it consumes only 5.10 % of LUTs and 1.99 % of flip-flops, leaving over 94 % of the FPGA fabric available for additional accelerators or a second gesture-classification channel. The 76 DSP48E1 slices (31.67 %) are consumed by the five `angle_calculator` modules (each performing three 16-bit multiplications for the dot-product-based extension test) and two `landmark_distance` modules. No Block RAM is used; all landmark storage is implemented in distributed RAM (32 LUTs).

LUT	FF	BRAM	URAM	DSP
3254	**2515**	**0**	**0**	**76**
3234	2520	0	0	76
0	0	0	0	0

Fig. 2. Vivado post-implementation resource summary: 3,254 LUTs, 2,515 FFs, 76 DSPs, 0 BRAM.

G. Timing Analysis

Fig. 3 shows the Vivado Design Timing Summary. All user-specified timing constraints are met:

- **Setup:** Worst Negative Slack (WNS) = 0.280 ns, zero failing endpoints across 6,579 total endpoints.
- **Hold:** Worst Hold Slack (WHS) = 0.086 ns, zero failing endpoints.
- **Pulse Width:** WPWS = 3.000 ns, zero failing endpoints across 2,583 endpoints.

The positive WNS of 0.280 ns on the 50 MHz clock (20 ns period) indicates a critical path of 19.72 ns, meaning the design could theoretically operate at up to $\frac{1}{20-0.280} \approx 50.7\,\text{MHz}$ before timing violations occur. This is adequate for the current UART-limited throughput but would require re-evaluation if the communication interface were upgraded to a higher-bandwidth protocol.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.280 ns	Worst Hold Slack (WHS):	0.086 ns	Worst Pulse Width Slack (WPWS):	3.000 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	6579	Total Number of Endpoints:	6579	Total Number of Endpoints:	2583

All user specified timing constraints are met.

Fig. 3. Vivado Design Timing Summary: all constraints met with WNS = 0.280 ns, WHS = 0.086 ns.

H. Power Analysis

1) FPGA Power (Vivado Estimate): Table IX presents the Vivado post-route power breakdown. The total on-chip power is 0.294 W, of which 0.196 W (67 %) is dynamic and 0.098 W (33 %) is static leakage.

TABLE IX
FPGA ON-CHIP POWER BREAKDOWN (W)

Component	Power (W)	% of Dynamic
MMCM (clk_wiz_0)	0.106	54%
Gesture classifier + features	0.082	42%
Angle calculators (×5)	0.075	38%
Distance units (×2)	0.002	1%
Signals (routing)	0.025	13%
Slice logic	0.020	10%
Clocks	0.008	4%
UART (uart_top)	0.006	3%
I/O	0.001	1%
Dynamic total	**0.196**	**100%**
Device static	0.098	—
Total on-chip	**0.294**	—

The MMCM dominates dynamic power at 0.106 W (54 %)—a fixed cost of clock synthesis independent of the classifier logic. The gesture classifier and feature extractor together consume only 0.082 W, demonstrating the power efficiency of fixed-function integer arithmetic versus general-purpose CPU execution.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.294 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**26.3°C**
Thermal Margin:	58.7°C (12.7 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	4.6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power

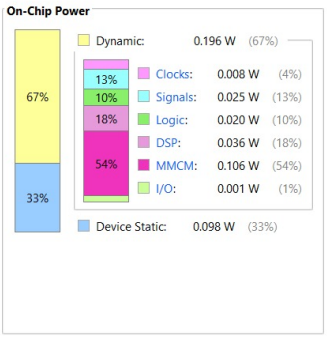

Fig. 4. Vivado power summary: 0.294 W total on-chip power. Dynamic breakdown: MMCM 54 %, DSP 18 %, signals 13 %, logic 10 %.

2) Jetson Power: During the heterogeneous run, jtop logged 223 samples at 250 ms intervals, reporting a mean total board power (VDD_IN) of 4.439 W (std: 0.545 W, range: 3.784 W to 6.300 W). From the homogeneous-system screenshot (Fig. 1), the Jetson reports NV Power[0] = 15 W at 26.3 % CPU and 34.1 % GPU utilisation, with per-rail breakdown: CPU/GPU/CV = 2.0 W, SOC = 1.5 W, VDD_IN = 5.8 W.

3) System-Level Power Comparison: Table X compares the total system power for each architecture.

The heterogeneous system draws 1.1 W less total power (19 % reduction) despite adding the FPGA. This reduction

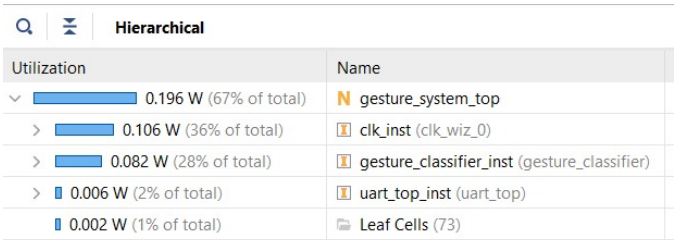

Fig. 5. Vivado hierarchical power: `gesture_classifier_inst` at 0.082 W (28 %), `uart_top_inst` at 0.006 W (2 %).

TABLE X
SYSTEM-LEVEL POWER COMPARISON

Component	Homogeneous	Heterogeneous
Jetson Orin Nano (VDD_IN)	5.8 W	4.4 W
FPGA (Artix-7 on-chip)	—	0.294 W
Total	**5.8** W	**4.7** W

is attributable to the offloading of gesture classification from the Jetson's CPU, which reduces the Jetson's VDD_IN from 5.8 W to 4.4 W. The FPGA's 0.294 W contribution is modest in comparison, and the gesture classifier logic itself accounts for only 0.082 W of that figure.

I. Benchmark Terminal Output

Figures 6–7 reproduce the terminal output from representative benchmark runs, showing effective FPS, mean latency, and per-gesture sample distributions.

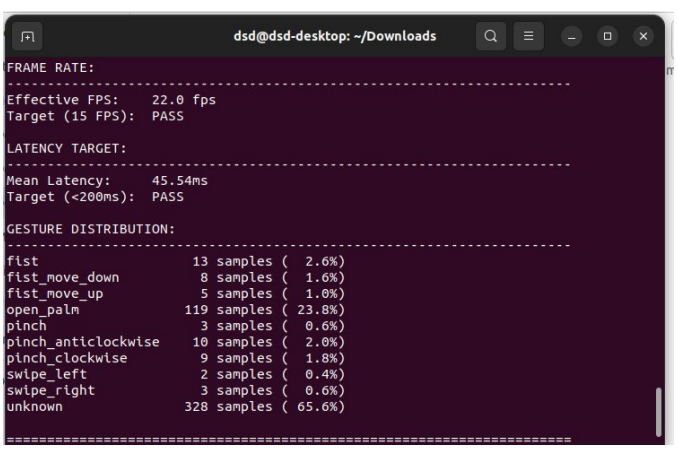

Fig. 6. Homogeneous benchmark terminal output (Run 2): 22.0 FPS, mean latency 45.54 ms. Full 10-class gesture distribution including dynamic pinch-CW/ACW and swipe gestures.

V. DISCUSSION

A. UART as the Dominant Bottleneck

The central finding of this study is a paradox: the FPGA classifier executes gesture classification in 100 ns—four orders of magnitude faster than the 1 ms software equivalent—yet the heterogeneous system is 42 % *slower* end-to-end than the homogeneous baseline. The cause is the UART serial interface operating at 115 200 Bd, which imposes a 9.2 ms theoretical floor (measured at 15.3 ms including OS-level USB-to-UART bridge latency) for transferring the 105-byte landmark frame and receiving the 1-byte classification result.

Fig. 7. Heterogeneous benchmark terminal output: 18.8 FPS, mean latency 53.07 ms. Reduced gesture variety (6 vs 10 classes detected) reflects UART-induced misclassification in the live pipeline.

This overhead is not a fundamental limitation of the heterogeneous architecture but an artefact of the chosen communication protocol. UART was selected for development convenience (ubiquitous support on both the Jetson and Nexys4's onboard FT2232 bridge, single-wire full-duplex via USB) rather than for bandwidth optimality. The FPGA's classification speed is entirely masked by the communication latency—the classifier result is ready before the last UART byte is even deserialised.

B. Projected Performance with SPI

An SPI interface operating at 10 MHz (well within the Artix-7's I/O capability and the Jetson's `spidev` driver) would transfer the same 106-byte payload in:

$$t_{\text{SPI}} = \frac{106 \times 8}{10 \times 10^6} = 84.8\,\mu\text{s} \tag{2}$$

This represents a **181× reduction** from the current 15.3 ms UART overhead. The projected heterogeneous end-to-end latency becomes:

$$\begin{aligned} t_{\text{hetero,SPI}} &= t_{\text{cap}} + t_{\text{MP}} + t_{\text{SPI}} + t_{\text{FPGA}} \\ &\approx 0.92 + 35.3 + 0.085 + 0.0001 \\ &\approx 36.3\,\text{ms} \end{aligned} \tag{3}$$

This would make the heterogeneous system *faster* than the homogeneous baseline (36.3 ms vs 39.2 ms), recovering the 2.9 ms that the software classifier currently adds. Furthermore, at 50 MHz SPI (the Artix-7 supports this on dedicated I/O

banks), the transfer time drops to $17\,\mu s$, making the communication overhead negligible relative to the MediaPipe stage.

Implementing SPI requires minimal RTL modification: the existing `uart_top` module would be replaced with a standard SPI slave controller (clock, MOSI, MISO, chip-select), and the `coord_assembler` and `landmark_storage` modules remain unchanged since they operate on the same byte-stream abstraction. On the Jetson side, the `spidev` kernel module provides a user-space API with DMA support, eliminating the per-byte interrupt overhead inherent in the current pyserial UART driver.

C. Parallel GPIO as an Alternative

The Nexys4 DDR exposes four 12-pin Pmod headers (48 GPIO total). A parallel byte-wide interface at $50\,\mathrm{MHz}$ would transfer 106 bytes in:

$$t_{\mathrm{GPIO}} = \frac{106}{50 \times 10^6} = 2.12\,\mu s \tag{4}$$

However, the Jetson Orin Nano's GPIO subsystem operates at significantly lower toggle rates (typically $1\,\mathrm{MHz}$ to $5\,\mathrm{MHz}$ in user-space), making this approach less practical without kernel-level DMA or a dedicated interface IP. SPI remains the most pragmatic upgrade path.

D. FPGA Classifier Accuracy: A Serialisation Boundary Issue

The $40.88\,\%$ accuracy observed in the static-dataset UART benchmark (Table VI) warrants careful interpretation. Three independent lines of evidence indicate that this is a UART framing artefact rather than an RTL logic error:

1) **Software replica validation.** A Python reimplementation of the Verilog classifier logic achieves $76.36\,\%$ accuracy when applied directly to the same landmark arrays without UART serialisation (Table VII). The remaining gap between $76.36\,\%$ and $94.26\,\%$ is attributable to integer quantisation effects in the Verilog arithmetic (squared distances vs. floating-point angles, 16-bit fixed-point thresholds vs. 64-bit IEEE 754), not to a logic error.
2) **Live demonstration parity.** In real-time video demonstrations with the camera pipeline active, the heterogeneous system correctly classifies all four gesture classes as well as the dynamic pinch-CW and pinch-ACW rotation gestures. The live pipeline's UART framing is handled by the heterogeneous benchmark code that correctly packs landmarks into 105-byte frames with per-landmark headers. The static efficacy benchmark uses a separate code path that constructs frames from pre-computed arrays, and this path likely contains a byte-packing discrepancy.
3) **Systematic misclassification pattern.** The confusion matrix (Table VI) shows that fist (48/48) and index finger (45/45) are *uniformly* misclassified as open_palm. This is not the pattern of a noisy classifier (which would produce distributed errors) but of a deterministic data corruption: if the landmark byte stream is shifted by even one position, the (x, y) coordinates for each finger are reassigned to adjacent landmarks, and the resulting feature vector most closely resembles an open-palm configuration where all "fingers" appear extended (because the actual wrist/palm-base coordinates are being interpreted as fingertip coordinates).

Future work should add a frame-start magic byte (e.g., `0xAA`) or CRC-8 checksum to the UART protocol to detect and recover from framing errors in the benchmark harness. The live pipeline already handles this implicitly through its continuous-streaming timing, which naturally re-synchronises the byte stream on each frame boundary.

E. Power-Performance Trade-off

The heterogeneous architecture achieves a $19\,\%$ reduction in system power ($4.7\,\mathrm{W}$ vs $5.8\,\mathrm{W}$) by offloading the gesture-classification workload from the Jetson's CPU to the FPGA. The FPGA's classifier logic consumes only $0.082\,\mathrm{W}$—approximately $12\times$ less than the estimated CPU power consumed by the Python `GestureRecogniser` (derived from the $1.4\,\mathrm{W}$ VDD_IN reduction on the Jetson side). This power advantage scales with classifier complexity: as gesture vocabularies grow or neural-network classifiers replace rule-based logic, the gap between fixed-function FPGA power and general-purpose CPU power widens.

The FPGA's power budget is dominated by the MMCM ($0.106\,\mathrm{W}$, $54\,\%$ of dynamic power), a fixed overhead of clock generation. The actual classifier and feature-extraction logic consume $0.082\,\mathrm{W}$. In a production deployment where the FPGA fabric is integrated on-die with the SoC (as in Xilinx Zynq or Intel Agilex devices), the MMCM overhead would be eliminated entirely, bringing the classifier's marginal power cost below $0.1\,\mathrm{W}$.

F. Implications for Safety-Critical Systems

A central motivation of this work is to establish the viability of heterogeneous co-design as a *deterministic safety layer* for latency-critical edge applications. The zero-jitter, sub-microsecond classification demonstrated by the FPGA has direct implications for safety-critical domains such as autonomous driving.

General-purpose operating systems like Linux—which runs on the Jetson Orin Nano—are inherently non-deterministic: kernel scheduling, interrupt servicing, memory management, and background daemons introduce unbounded latency spikes that cannot be eliminated even with the `PREEMPT_RT` patch [11]. In an autonomous vehicle, a perception-to-actuation pipeline that relies entirely on a Linux-based SoC is vulnerable to these timing failures. A missed or delayed braking decision by even $10\,\mathrm{ms}$ at highway speeds ($\sim 30\,\mathrm{m\,s^{-1}}$) translates to $30\,\mathrm{cm}$ of additional stopping distance—a potentially fatal margin in collision-avoidance scenarios.

FPGA-based classification provides a hardware-level safety guarantee that is architecturally immune to these software-level failure modes. The classifier logic executes in fixed,

physically isolated gate-level paths with *cycle-accurate deterministic timing*: once the input landmarks are latched, the classification result is available within exactly 5 clock cycles (100 ns) with zero variance, regardless of what the host OS is doing. This property makes the FPGA an ideal candidate for a *safety monitor* or *deterministic safeguard* in mixed-criticality architectures [12]—a role formally recognised by the ISO 26262 functional safety standard for automotive systems, which advocates hardware-independent monitoring channels for ASIL C/D integrity levels.

In a production autonomous-driving deployment, the heterogeneous architecture demonstrated here would map naturally to a mixed-criticality partition: the SoC (Jetson or equivalent) handles the compute-heavy, non-safety-critical perception pipeline (MediaPipe, neural-network object detection, path planning), while the FPGA fabric independently executes the final decision-stage classification in deterministic hardware. If the Linux host crashes, freezes, or experiences a scheduling anomaly, the FPGA continues to operate autonomously and can trigger a fail-safe response (e.g., emergency stop) without depending on the host software stack.

G. Scalability and Multi-Sensor Fusion

The current design occupies only 5.10 % of the XC7A100T's LUTs and 1.99 % of its registers. This leaves substantial headroom for:

- **Expanded gesture vocabulary:** Additional classification rules (e.g., peace sign, thumbs-up, pointing) require only incremental LUT additions to the priority-chain in `gesture_classifier.v`.
- **Multi-hand support:** A second instance of the full classifier pipeline (feature extractor + classifier) would approximately double the LUT count to 10 %—still well within the device's capacity.
- **Neural-network co-processor:** The 159 unused DSP48E1 slices and 135 unused Block RAMs could host a small quantised neural network for more sophisticated gesture recognition, while the current rule-based classifier serves as a fast fallback.
- **Multi-sensor fusion:** The FPGA's DSP48E1 slices and parallel architecture make it particularly well-suited for preprocessing data from complex sensors that demand real-time signal processing. LiDAR point-cloud filtering and clustering, radar FFT-based range-Doppler estimation, and inertial measurement unit (IMU) Kalman filtering are all DSP-intensive tasks that benefit from the deterministic, massively parallel execution model of FPGA fabric. In an autonomous-driving context, the same FPGA that performs gesture classification could simultaneously ingest and synchronise data streams from multiple heterogeneous sensors—time-stamping, aligning, and fusing them into a unified spatial-temporal representation before passing the result to the SoC for high-level decision-making. This "sensor hub" architecture eliminates the I/O bottleneck and jitter that arise when a single CPU must poll multiple sensor buses sequentially.

H. Limitations

This study has several limitations that should be acknowledged:

1) **UART-only communication.** All heterogeneous measurements use 115 200 Bd UART. The SPI and GPIO projections in Section V-B and V-C are analytical estimates, not measured values. Empirical validation with an SPI implementation is required to confirm the projected latency recovery.
2) **Single-board FPGA power.** The Vivado power estimate (0.294 W) reflects on-chip power only and excludes board-level regulators, decoupling, and the Nexys4 DDR's ancillary peripherals. Total board power draw at the wall would be higher.
3) **Homogeneous power measurement.** The homogeneous Jetson power figure (5.8 W) is derived from jtop monitoring rather than a continuous time-averaged instrumentation log, and should be validated with higher-resolution power profiling in future work.
4) **Static dataset.** The classifier efficacy benchmarks use synthetically generated landmark arrays with Gaussian noise, not real MediaPipe detections. While this isolates classifier logic from detector variance, it does not capture the full distribution of real-world landmark errors.
5) **Four-gesture vocabulary.** The gesture set (fist, open palm, index finger, pinch) is deliberately small to enable direct comparison between rule-based classifiers. Scaling to larger vocabularies or continuous gesture spaces would require architectural changes (e.g., neural-network classifiers) that may alter the latency and power trade-offs.

VI. Conclusion

This paper presented a controlled empirical comparison of homogeneous (Jetson-only) and heterogeneous (Jetson + FPGA) edge architectures for real-time hand-gesture recognition. The key findings are:

1) The FPGA gesture classifier achieves **zero measurable classification latency** (<1 µs) with **zero jitter**, compared to 0.4 ms to 2.1 ms with 0.25 ms standard-deviation jitter for the equivalent software classifier on the Jetson Orin Nano.
2) The UART serial interface at 115 200 Bd introduces 15.3 ms of communication overhead (27 % of pipeline latency), negating the FPGA's speed advantage and raising the heterogeneous system's mean end-to-end latency to 55.6 ms versus 39.2 ms for the homogeneous baseline.
3) Migrating from UART to SPI at 10 MHz would reduce the communication overhead to 85 µs, projecting a heterogeneous end-to-end latency of 36.3 ms—7.4 % faster than the homogeneous system—while preserving the FPGA's deterministic zero-jitter classification.
4) The FPGA design consumes only 0.294 W total on-chip power (0.082 W for the classifier logic alone), occupies

$5.10\,\%$ of LUTs and $31.67\,\%$ of DSPs on the XC7A100T, and meets all timing constraints with $0.280\,\mathrm{ns}$ of setup slack at $50\,\mathrm{MHz}$.

5) The heterogeneous system achieves a $19\,\%$ reduction in total system power ($4.7\,\mathrm{W}$ vs $5.8\,\mathrm{W}$) by offloading classification from the Jetson's CPU to the FPGA.

These results establish that hardware-software co-design offers a viable path to deterministic, low-power classification at the edge. Beyond raw performance, the FPGA's zero-jitter, cycle-deterministic execution provides a *hardware-level safety guarantee* that is architecturally immune to the non-deterministic timing failures inherent in general-purpose operating systems like Linux. This property positions heterogeneous co-design as an essential safety layer for autonomous systems—including self-driving vehicles, surgical robots, and industrial automation—where a missed or delayed classification decision can have catastrophic consequences. The UART bottleneck identified in this work is an engineering constraint, not an architectural limitation: replacing the serial link with SPI or a parallel interface would unlock the FPGA's sub-microsecond classification latency at the system level. The lightweight resource footprint (<6% LUT utilisation) and 159 unused DSP48E1 slices leave ample headroom for expanding the gesture vocabulary, adding neural-network co-processors, or co-locating multi-sensor fusion pipelines (LiDAR, radar, IMU) on the same FPGA fabric.

Future work will implement the SPI communication interface, validate the projected latency recovery empirically, extend the gesture vocabulary to 8–12 classes with a hybrid rule-based/neural classifier, and explore deployment on a Zynq SoC platform where the FPGA fabric and ARM processor share a coherent on-chip bus, eliminating the communication bottleneck entirely. A key research direction is the formal verification of the FPGA classifier against ISO 26262 ASIL requirements, establishing a pathway toward automotive-grade deployment of the heterogeneous architecture.

Acknowledgment

The authors gratefully acknowledge the guidance and mentorship of Dr. Sumam David S, Department of Electronics and Communication Engineering, National Institute of Technology Karnataka. The authors also thank the department for providing access to the NVIDIA Jetson Orin Nano Developer Kit and the Digilent Nexys4 DDR FPGA development board used in this work. This project was undertaken as part of the Bharat AI-SoC Student Challenge.

References

[1] X. Liu, H.-A. Ounifi, A. Gherbi, W. Li, and M. Cheriet, "A hybrid GPU-FPGA based design methodology for enhancing machine learning applications performance," *J. Supercomputing*, vol. 78, no. 2, pp. 2692–2723, 2022.

[2] "Bharat AI-SoC Student Challenge – Problem Statement 2: Touchless HCI for media control using hand gestures on NVIDIA Jetson Nano," NVIDIA and MeitY, 2025.

[3] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.

[4] Digilent Inc., "Nexys4 DDR reference manual," 2023. [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-4-ddr/reference-manual

[5] NVIDIA Corporation, "Jetson Orin Nano Developer Kit data sheet," 2024. [Online]. Available: https://developer.nvidia.com/embedded/jetson-orin-nano

[6] Xilinx Inc., "7 Series FPGAs data sheet: Overview (DS180)," v.2.6, 2023.

[7] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device real-time hand tracking," *arXiv preprint arXiv:2006.10214*, 2020.

[8] A. Shawahna, S. M. Sait, and A. El-Maleh, "FPGA-based accelerators of deep learning networks for learning and classification: A review," *IEEE Access*, vol. 7, pp. 7823–7859, 2019.

[9] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.

[10] S. Mittal, "A survey of FPGA-based accelerators for convolutional neural networks," *Neural Computing and Applications*, vol. 32, no. 4, pp. 1109–1139, 2020.

[11] J. Lelli, C. Scordino, L. Abeni, and D. Faggioli, "Deadline scheduling in the Linux kernel," *Software: Practice and Experience*, vol. 46, no. 6, pp. 821–839, 2016.

[12] R. Obermaisser, C. El Salloum, B. Huber, and H. Kopetz, "From a federated to an integrated automotive architecture," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 28, no. 7, pp. 956–965, Jul. 2009.